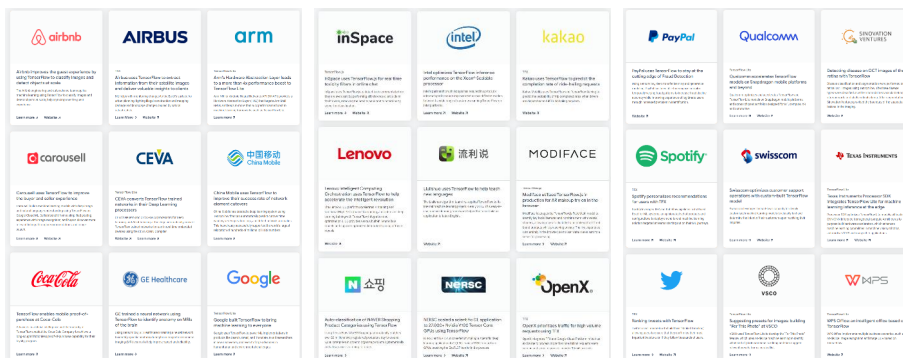


# [Chap 9] Tensorflow 기초와 회귀분석



## Tensorflow

- 구글에서 배포하고 있는 오픈소스 머신러닝 플랫폼
  - 머신러닝을 위한 수치 연산 패키지 집합
- 다양한 기업에서 활용 ([www.tensorflow.org](http://www.tensorflow.org))



- 인공지능망 모델링을 위한 케라스(Keras) 라이브러리와 연동
  - 구글에서 함께 배포

# Tensorflow 설치

- pip install tensorflow (MacOS: pip3 install tensorflow)

```
C:\WINDOWS\system32>pip install tensorflow
Collecting tensorflow
  Downloading tensorflow-2.9.1-cp39-cp39-win_amd64.whl (444.0 MB)
    | 203.5 MB 6.4 MB/s eta 0:00:38_
```

- 기본 설치 상태에서는 CPU에서 동작
  - 큰 문제에 활용하기 위해서는 GPU(그래픽카드)의 활용이 필요

3

## Tensorflow의 기본 데이터 단위 Tensor

- Tensor 상수의 정의

```
1 import tensorflow as tf
2 a = tf.constant(1)
3 print(a)
```

```
tf.Tensor(1, shape=(), dtype=int32)
```

- (1행) 텐서플로를 tf라는 별명으로 불러오기
- (2행) 숫자 1을 값으로 갖는 텐서 상수를 정의
  - 자료형을 선언하지 않았으므로 자동으로 정수로 설정됨
    - 출력 결과에서 dtype=int32

4

# Tensor의 자료형 선언

- 자료형을 명시적으로 실수로 정의하기
  - 실수로 계산할 데이터는 처음부터 명시적으로 실수로 선언하는 것이 바람직

```
1 import tensorflow as tf
2 a = tf.constant(1, dtype=tf.float32)
3 print(a)
```

```
tf.Tensor(1.0, shape=(), dtype=float32)
```

- 텐서플로우의 자료형 (표 9-1)

| 텐서플로우 자료형  | 의미                      | 값의 표현 범위                                           |
|------------|-------------------------|----------------------------------------------------|
| tf.float32 | 32비트 실수                 | -3.4028235e+38 ~ 3.4028235e+38                     |
| tf.float64 | 64비트 실수                 | -1.7976931348623157e+308 ~ 1.7976931348623157e+308 |
| tf.int8    | 8비트 정수                  | -128 ~ 127                                         |
| tf.int16   | 16비트 정수                 | -32768 ~ 32767                                     |
| tf.int32   | 32비트 정수                 | -2,147,483,648 ~ 2,147,483,647                     |
| tf.uint8   | 8비트 비음(nonnegative) 정수  | 0 ~ 255                                            |
| tf.uint16  | 16비트 비음(nonnegative) 정수 | 0 ~ 65535                                          |
| tf.string  | 문자열                     | 모든 문자열                                             |
| tf.bool    | 논리값 (True 또는 False)     | True, False                                        |

5

## Tensor에 행렬 저장

- 행렬을 저장하기 위해서는 행을 [ ]로 감싸고 전체를 다시 [ ]로 묶음
  - Numpy와 유사한 구조

- 2행 2열 행렬의 예

```
1 import tensorflow as tf
2 a = tf.constant([[3.14, 1.7], [5.6, 2.2]], dtype=tf.float32)
3 print(a)
```

– shape=(2, 2) 출력

- 3행 2열 행렬의 예

```
1 import tensorflow as tf
2 a = tf.constant([[3.14, 1.7], [5.6, 2.2], [3, 3]], dtype=tf.float32)
3 print(a)
```

– shape=(3, 2) 출력

6

# Tensorflow를 이용한 행렬의 덧셈과 곱셈

- 두 행렬의 합과 곱을 출력

```
1 import tensorflow as tf
2 a = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
3 b = tf.constant([[2, 2], [3, 5]], dtype=tf.float32)
4 print(a+b)
5 print(tf.matmul(a, b))
```

```
tf.Tensor(
[[3.  4.]
 [6.  9.]], shape=(2, 2), dtype=float32)
tf.Tensor(
[[ 8. 12.]
 [18. 26.]], shape=(2, 2), dtype=float32)
```

7

## Tensorflow의 단위행렬과 역행렬

- 2행 2열의 단위행렬 생성 및 연산

```
1 import tensorflow as tf
2 a = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
3 i = tf.constant(tf.eye(2), dtype=tf.float32)
4 print(tf.matmul(a, i))
5 print(tf.matmul(i, a))
```

- (3행) 2행 2열 단위행렬을 생성하여 변수 i에 저장
- (4~5행) a에 단위행렬을 곱한 값을 출력: 원래 행렬이 변하지 않음

- 역행렬(inverse matrix) 계산

```
1 import tensorflow as tf
2 a = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
3 aInv = tf.linalg.inv(a)
4 print(aInv)
5 print(tf.matmul(a,aInv))
```

- (3행) a의 역행렬을 계산하여 aInv에 저장
- (5행) a와 aInv를 곱하면 단위행렬 출력
  - 컴퓨터의 소수점 한계로 인해 약간의 오차 발생 가능

8

# 회귀분석의 원리

- 회귀분석(regression)
  - 인과적 예측모형의 대표적 방법론
- 머신러닝(machine learning)의 본질은 데이터를 설명하는 모델을 수립하는 것
  - 모델의 틀을 잡고 데이터를 부어 넣어 틀의 모양을 데이터에 맞게 다듬어 가는 과정
  - 모델의 틀 안에는 모델의 세부 모양을 결정하는 파라미터(parameter)들이 있음
  - 머신러닝은 데이터들을 가장 잘 설명하기 위한 파라미터의 값을 찾는 것
    - 이 과정을 학습(training), 피팅(fitting), 또는 최적화(optimization)라고 함
- 머신러닝이란 결국, 모델의 파라미터를 최적화함으로써 모델이 예측하는 값과 실제 정답값 사이의 차이를 최소화하는 것이라고 할 수 있음
- 회귀분석은 머신러닝의 하나로 이해할 수 있음
  - 회귀분석을 텐서플로를 이용해 구현함으로써 머신러닝의 구조 이해 가능

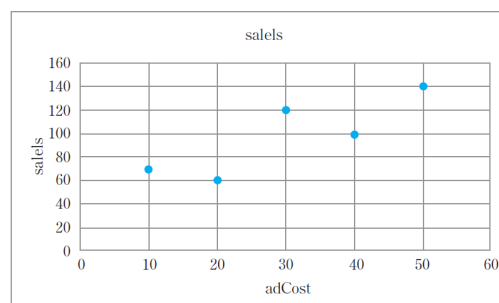
9

## 회귀분석 예제

- 광고료(adCost)와 매출(sales)의 관계
  - 광고료를 많이 쓸수록 매출이 증가하는 경향

〈표 9-2〉 회귀분석 예제

| adCost | sales |
|--------|-------|
| 30     | 120   |
| 20     | 60    |
| 50     | 140   |
| 40     | 100   |
| 10     | 70    |



[그림 9-3] 광고료와 매출의 산점도

- 이 관계를  $y=ax+b$ 의 직선식의 틀로 설명하고자 함
  - adCost를  $x$ , sales를  $y$ 로 둠
  - 이 틀에는 두 개의 파라미터  $a$ ,  $b$ 가 있음
  - $x$ 에 광고료를 넣었을 때 계산된  $y$ 와 실제 매출액 사이의 차이가 가장 적어지도록  $a$ ,  $b$ 의 값을 최적화하는 것이 목적

# 파라미터 a, b의 최적화

- 처음에는 a, b에 아무 값이나 대입해 봄 (a=2, b=40)
  - $\hat{y} = 2x + 40$

〈표 9-3〉 초기모델의 예측치와 실제값의 비교

| x  | y   | $\hat{y}$ | $y - \hat{y}$ |
|----|-----|-----------|---------------|
| 30 | 120 | 100       | 20            |
| 20 | 60  | 80        | -20           |
| 50 | 140 | 140       | 0             |
| 40 | 100 | 120       | -20           |
| 10 | 70  | 60        | 10            |

오차  
(error)

- 오차( $y - \hat{y}$ )를 최소화하는 것이 목적
  - 오차 총량 지표를 만들어 이 값을 줄여 나가고자 함
  - 오차의 단순합은 +, - 가 상쇄되므로 오차 지표로서 부적합
  - 오차 제곱합(SSE) 또는 평균제곱오차(MSE)를 많이 사용
- 현재의 MSE는  $(20^2 + (-20)^2 + 0^2 + (-20)^2 + 10^2)/5 = 260$ 
  - a와 b를 조정하여 이 값을 줄여야 함

11

## 파라미터 a, b의 최적화 (계속)

- 임의의 a, b에 대한 MSE의 계산식

$$\begin{aligned}
 MSE &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - (ax_i + b))^2 \\
 &= \frac{1}{5} \left\{ (120 - (30a + b))^2 + (60 - (20a + b))^2 + (140 - (50a + b))^2 \right. \\
 &\quad \left. + (100 - (40a + b))^2 + (70 - (10a + b))^2 \right\} \quad (\text{식 9-1}) \\
 &= 1100a^2 + b^2 + 60ab - 6600a - 196b + 10500
 \end{aligned}$$

- a=2, b=40을 대입해 보면 MSE=260이 나옴을 확인
- 미분(differentiation)의 개념
  - 어떤 변수의 특정 값에서 함수의 순간적인 기울기
    - 어떤 변수가 아주 조금 증가할 때 해당 함수가 얼마나 증가하는지를 알려줌
    - 순간기울기를 그래디언트(gradient)라고 함
- 만약 a에 대한 MSE 함수의 그래디언트가 + 라면, a가 증가하면 MSE가 증가한다는 뜻이 되므로, MSE를 줄이려면 a를 약간 감소시켜야 할 것임
  - 반대로 그래디언트가 - 라면 a를 조금 증가시켜야 MSE가 감소할 것임

## 파라미터 a, b의 최적화 (계속)

- MSE를 a 및 b에 대해 미분하면 다음과 같이 나타남

$$\frac{\partial MSE}{\partial a} = 2200a + 60b - 6600 \quad (\text{식 9-3})$$

$$\frac{\partial MSE}{\partial b} = 60a + 2b - 196 \quad (\text{식 9-4})$$

- 위 식에 a=2, b=40을 대입해 보면 각각 200, 4가 계산됨
  - a에 대한 그래디언트가 200으로 양수이므로, a를 약간 감소시켜야 MSE가 감소
  - b에 대한 그래디언트가 4로 양수이므로, b를 약간 감소시켜야 MSE가 감소
- 예로서, a와 b를 각각 0.01씩 감소시켜서 MSE를 다시 계산해 봄
  - (식 9-1)에 a=1.99, b=39.99를 대입하면 MSE = 258.076으로 이전보다 감소하였음
- 다시 a=1.99, b=39.99를 (식 9-3)와 (식 9-4)에 대입
  - a, b의 그래디언트가 각각 177.4, 3.38로 계산
  - 둘 다 양수이므로 a, b를 조금 더 감소시켜야 함
- 이와 같은 과정을 반복하여 a, b에 대한 그래디언트가 0에 다다르면 최적해에 도달

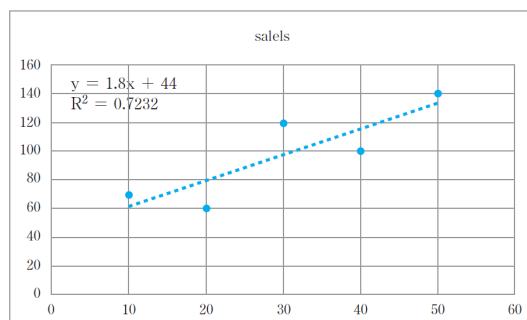
13

## 회귀식 도출의 다양한 방법

- 수학적으로 최적 a, b를 한 번에 구하는 방법
  - (식 9-3)과 (식 9-4)를 모두 0으로 두고 연립방정식의 해를 구하면 됨

$$\begin{aligned} 2200a + 60b - 6600 &= 0 \\ 60a + 2b - 196 &= 0 \end{aligned} \quad (\text{식 9-5})$$

- a=1.8, b=44가 도출됨
- 엑셀의 추세선 기능 활용
  - $R^2 = 0.7232$  : 이 모델이 데이터 변동의 약 72% 정도를 설명한다는 의미



[그림 9-4] 엑셀의 추세선 기능을 활용한 회귀분석

# 회귀식 도출의 다양한 방법 (계속)

- 파이썬으로 회귀식 도출
  - statsmodels 설치: `pip install statsmodels`
  - OLS 함수로 선형회귀 수행

```
1 x = [30,20,50,40,10]
2 y = [120,60,140,100,70]
3 import statsmodels.api as sm
4 results = sm.OLS(y, sm.add_constant(x)).fit()
5 print(results.summary())
```

## OLS Regression Results

```
=====
Dep. Variable:          y      R-squared:          0.723
Model:                  OLS    Adj. R-squared:      0.631
Method:                 Least Squares      F-statistic:      7.839
Date:                   Wed, 18 May 2022    Prob (F-statistic):    0.0679
Time:                   01:47:56    Log-Likelihood:      -20.878
No. Observations:       5      AIC:              45.76
Df Residuals:           3      BIC:              44.98
Df Model:               1
Covariance Type:        nonrobust
=====
```

|       | coef    | std err | t     | P> t  | [0.025  | 0.975]  |
|-------|---------|---------|-------|-------|---------|---------|
| const | 44.0000 | 21.323  | 2.064 | 0.131 | -23.859 | 111.859 |
| x1    | 1.8000  | 0.643   | 2.800 | 0.068 | -0.246  | 3.846   |

```
=====
```

15

## 회귀분석과 머신러닝

- 회귀분석 모델의 파라미터는 계산을 통해, 또는 패키지를 이용해 쉽게 구할 수 있으나, 미분을 통해 파라미터를 조금씩 수정해 나가는 과정이 머신러닝의 작동 방식과 유사
- 머신러닝은 모델의 틀을 수립해 놓고 파라미터를 초기값으로 설정한 후, 개선방향을 찾고, 파라미터를 수정하고, 다시 개선방향을 찾는 과정의 반복으로 이루어짐
  - 텐서플로우로 이 과정을 구현해 보고자 함

16

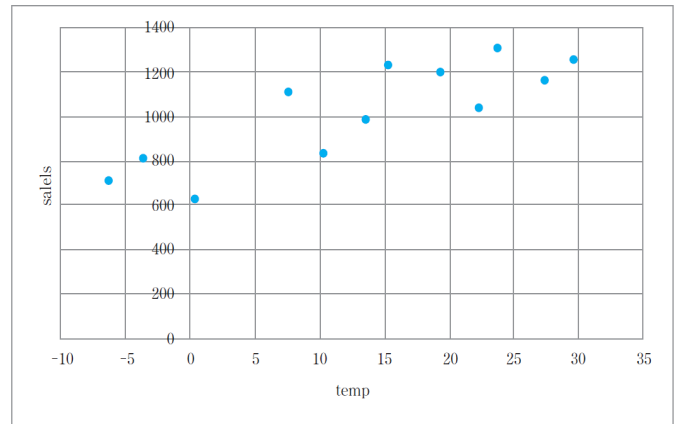


# 예제 데이터

- softdrink.csv
  - 어떤 음료수 회사의 월별 평균기온과 매출액 사이의 관계
  - 우상향하는 선형성: 기온이 높아질수록 음료수가 많이 팔리는 패턴
  - 선형 회귀분석 모델: 기온을  $x$ , 매출액을  $y$ 로 두고  $y=ax+b$ 의 관계를 전제

<표 9-4> 월별 평균기온과 매출액 데이터

| month (월) | temp (도) | sales (백만원) |
|-----------|----------|-------------|
| 1         | -6.3     | 714         |
| 2         | -3.6     | 810         |
| 3         | 10.2     | 836         |
| 4         | 15.3     | 1233        |
| 5         | 19.4     | 1204        |
| 6         | 22.3     | 1037        |
| 7         | 29.7     | 1258        |
| 8         | 27.3     | 1161        |
| 9         | 23.7     | 1310        |
| 10        | 13.5     | 986         |
| 11        | 7.6      | 1109        |
| 12        | 0.4      | 634         |



[그림 9-5] 기온과 매출액 관계 산점도

17

## Tensorflow로 회귀분석 구현

- 데이터 읽어오기

```
1 import pandas as pd
2 sd = pd.read_csv('softdrink.csv')
```

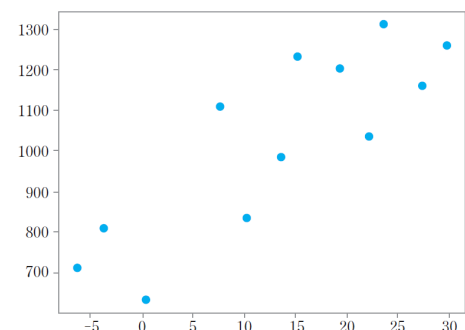
- 데이터를 텐서에 저장

- temp(기온)를  $x$ 에, sales(매출액)를  $y$ 에 저장

```
1 import tensorflow as tf
2 x = tf.constant(sd['temp'], dtype=tf.float32)
3 y = tf.constant(sd['sales'], dtype=tf.float32)
4 print(x)
5 print(y)
```

- 산점도 출력

```
1 import matplotlib.pyplot as plt
2 plt.scatter(x, y, color='blue')
3 plt.show()
```



[그림 9-6] 기온과 매출액의 관계를 나타내는 산점도

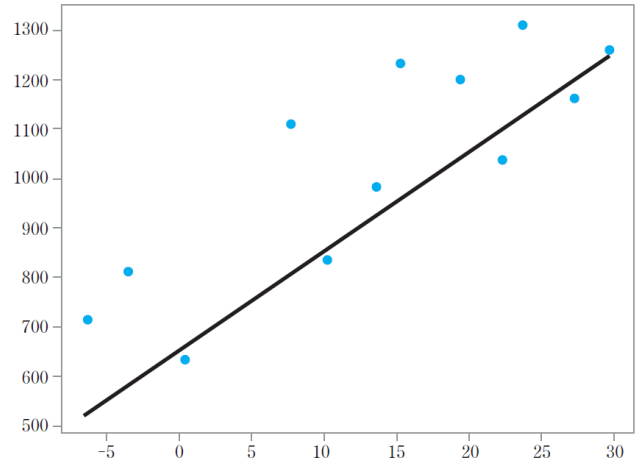
18

# 초기 모델 수립

- 파라미터 a, b의 초기값을 설정하고 예측값을 계산하여 산점도를 출력
  - (8~9행) a, b의 초기값을 각각 20, 650으로 설정
  - (10행) 현재의 모델로 예측값  $y_{\text{hat}}$ 을 계산
  - (12~15행) x에 대해 y 및  $y_{\text{hat}}$ 의 산점도를 출력 → 현재 모델의 설명력이 좋지 않음을 확인

예제 ex9-1.py

```
1 import pandas as pd
2 sd = pd.read_csv('softdrink.csv')
3 import tensorflow as tf
4 x = tf.constant(sd['temp'], dtype=tf.float32)
5 y = tf.constant(sd['sales'], dtype=tf.float32)
6 print(x)
7 print(y)
8 a = tf.Variable(20, dtype=tf.float32)
9 b = tf.Variable(650, dtype=tf.float32)
10 y_hat = a*x+b
11 print(y_hat)
12 import matplotlib.pyplot as plt
13 plt.scatter(x, y, color='blue')
14 plt.plot(x, y_hat, color='red')
15 plt.show()
```



[그림 9-7] 초기 예측모델

19

## 오차 측정

- 확장성을 위해 예측값  $y_{\text{hat}}$  계산을 함수로 정의

```
1 def y_hat(x, a, b):
2     return a*x + b
```

- 함수  $y_{\text{hat}}$ 을 이용하여 초기 모델의 예측값 출력

```
1 import pandas as pd
2 import tensorflow as tf
3
4 def y_hat(x, a, b):
5     return a*x + b
6
7 sd = pd.read_csv('softdrink.csv')
8 x = tf.constant(sd['temp'], dtype=tf.float32)
9 y = tf.constant(sd['sales'], dtype=tf.float32)
10 a = tf.Variable(20, dtype=tf.float32)
11 b = tf.Variable(650, dtype=tf.float32)
12 print(y_hat(x,a,b))
```

```
tf.Tensor([ 524.  578.  854.  956. 1038. 1096. 1244. 1196. 1124.  920.  802.  658.],
shape=(12,), dtype=float32)
```

20

# 오차 측정 (계속)

- 예측값과 실제값 사이의 평균제곱오차(MSE)를 계산하는 함수 loss를 정의

```
1 def loss(a, b, x, y):  
2     return tf.reduce_mean(tf.square(y - y_hat(x, a, b)))
```

- tf.reduce\_mean : 벡터의 개별 값들에 대한 평균을 구해주는 텐서플로우 함수

- 초기모델 예측값의 오차 계산

- MSE=27767.666으로 계산됨
- 오차를 줄여 나가야 함 (최적화)

예제 ex9-2.py

```
...  
10 sd = pd.read_csv('softdrink.csv')  
11 x = tf.constant(sd['temp'], dtype=tf.float32)  
12 y = tf.constant(sd['sales'], dtype=tf.float32)  
13 a = tf.Variable(20, dtype=tf.float32)  
14 b = tf.Variable(650, dtype=tf.float32)  
15  
16 print('MSE = ', loss(a,b,x,y))
```

MSE = tf.Tensor(27767.666, shape=(), dtype=float32)

21

# 그래디언트 계산

- 오차를 줄이기 위해 a, b 각각에 대한 그래디언트 값을 구함
  - 그래디언트의 부호가 +이면 파라미터를 조금 감소시키고, -이면 조금 증가시켜야 함
- 텐서플로우의 미분 계산
  - GradientTape을 이용하여 미분 대상인 함수와 텐서 변수들을 기억
    - 파이썬의 with.. as.. 구문과 함께 사용
  - gradient 함수로 미분 계산
- 파라미터 a,b에 대한 그래디언트 계산

```
18 with tf.GradientTape() as t:  
19     current_loss=loss(a, b, x, y)  
20  
21 da, db = t.gradient(current_loss, [a,b])  
22 print('da = ', da)  
23 print('db = ', db)
```

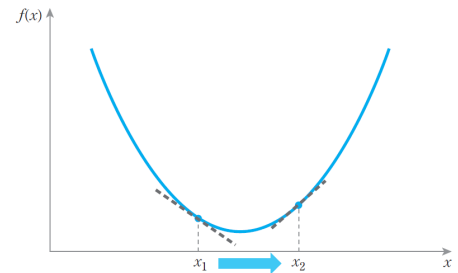
MSE = tf.Tensor(27767.666, shape=(), dtype=float32)  
da = tf.Tensor(-1835.0167, shape=(), dtype=float32)  
db = tf.Tensor(-217.00002, shape=(), dtype=float32)

- (18~19행) GradientTape에 미분하려는 함수 연산을 기록
- (21행) da = a에 대한 그래디언트, db = b에 대한 그래디언트 저장
- da, db가 모두 음수로 나타남

22

# 모델의 최적화: 경사하강법

- $da, db$  값이 모두 음수이므로, loss 함수를 감소시키기 위해서는  $a, b$ 를 증가시켜야 함
- 값을 한번에 많이 변경하는 것은 위험
  - 그래디언트는 순간기울기이므로, 값이 많이 바뀌면 바뀐 값에서도 기울기가 동일하다는 보장이 없음
  - 그림 9-8에서  $x_1$ 에서  $x_2$ 로 한꺼번에 가버리면 오히려 함수 값이 증가하고 기울기는 반대여서 되돌아와야 하는 상황
- 학습률(learning rate)
  - 파라미터를 한 번에 변화시키는 정도
  - 학습률을 그래디언트 값에 곱한 값만큼 값을 변화시킴
    - 예) 학습률 0.001 : 한 번에 그래디언트의 0.1%만큼 증가 또는 감소
- 학습률을 0.001로 설정하고  $a, b$ 를 개선



[그림 9-8] 순간기울기와 학습률

```
1 a.assign_add(-0.001*da)
2 b.assign_add(-0.001*db)
```

- 이 과정을 그래디언트 값이 0이 될 때까지 반복
- 소수점 오차로 완전히 0이 되지 않을 수 있으므로, 그래디언트가 일정 값 이하이면 종료

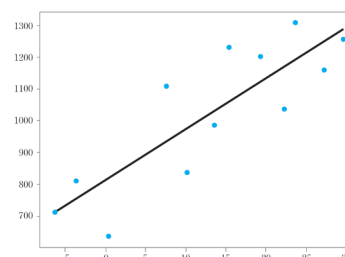
**경사하강법(Gradient Descent Method)**  
그래디언트의 부호를 기반으로  $x$ 값을 조금씩 변화시켜 손실을 줄여나가는 최적화 기법

23

## Tensorflow를 활용한 회귀분석 결과

- Tensorflow로 경사하강법 구현
  - (17~18행)  $a, b$ 에 대한 loss 함수의 그래디언트를 구하여  $da, db$ 에 저장
  - (23~24행) 학습률 0.001을 적용하여  $a, b$ 를 개선
  - (26행)  $da, db$ 가 모두 절대값 0.05 미만이면 최적해 도달 판단, while 반복 종료
  - (28~31행)  $x$ 에 대한 정답값  $y$ 와 최종 모델의 예측값  $y_{hat}$ 의 산점도 출력
- 출력 결과
  - MSE는 13896.519까지 감소
    - 최종  $a=16.00, b=811.62$ 로 계산됨
  - 패키지를 이용한 정확한  $a, b$ 는 각각 15.999, 811.68로 거의 비슷한 값 산출

```
예제 ex9-4.py
...
16 while True:
17     with tf.GradientTape() as t:
18         current_loss=loss(a, b, x, y)
19         da, db = t.gradient(current_loss, [a,b])
20         print('MSE, a, b, da, db = ',
21               a.numpy(), b.numpy(), loss(a,b,x,y).numpy(), da.numpy(), db.numpy())
22
23         a.assign_add(-0.001*da)
24         b.assign_add(-0.001*db)
25
26         if abs(da)<0.05 and abs(db)<0.05: break
27
28 import matplotlib.pyplot as plt
29 plt.scatter(x, y, color='blue')
30 plt.plot(x, y_hat(x,a,b), color='red')
31 plt.show()
```



[그림 9-9] 최종 모델의 시각화

24

# 머신러닝과 비선형최적화

---

- 회귀분석의 오차함수로 MSE를 사용하였음
  - MSE는 2차항이 포함된 비선형 함수(nonlinear function)
  - 이 함수를 경사하강법으로 최적화하였음
- 비선형 최적화(nonlinear optimization): 머신러닝의 근간
  - 경사하강법은 비선형 최적화 방법 중 하나
- 학습률의 중요성
  - 문제 특성에 따라 학습률의 조정이 필요
  - 학습률을 처음에는 빠르게, 최적점 근처에서는 느리게 하는 등 유연하게 조절하기도 함